Load datasets

- Use the two IoT datasets: **CICIoT2023** (47 features, 8 classes) and **IoTID20** (83 features, 5 classes).
- (Paper split): train : val : test = **70 : 10 : 20**.

✅ What are features?

Features are the input variables (columns) that describe each data sample.

They are the measurable attributes that the model uses to learn patterns.

Examples of features in IoT intrusion datasets might include:

✅ What are classes?

Classes are the labels (target outputs) you want the model to predict.

They represent the categories or attack types.

Examples:

- Benign
- DoS attack
- DDoS attack
- Ransomware
- Mirai
- Scan
- Injection attack
- etc.

So:

✨ Summary

Dataset	# Features	# Classes	Meaning	
CICIoT2023	47	8	47 input variables, 8 possible attack/benign labels	
IoTID20	83	5	83 input variables, 5 possible attack/benign labels	

Replace categorical target labels with integers.

- CICIoT2023 `label` → e.g. {BruteForce:0, DDoS:1, DoS:2, Mirai:3, Normal:4, Recon:5, Spoofing:6, Web-Based:7}.
- IoTID20 `Sub Cat` → 5 integers (0..4).

Optional input preprocessing

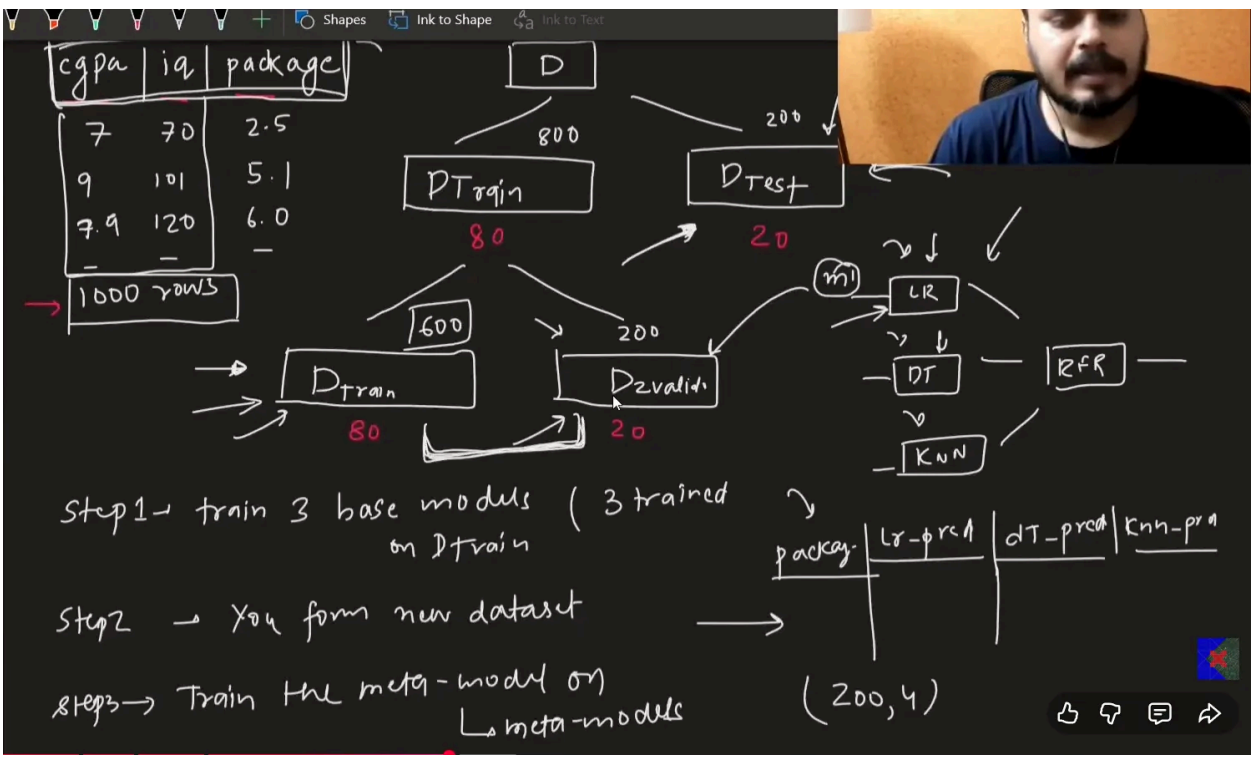
- Handle missing values, normalize/scale features if needed (tree models usually don't require scaling).
- Encode any categorical input features (one-hot or ordinal as appropriate).

Doubt ::

Feature selection using MDI (Mean Decrease in Impurity)

- Train a tree-based model (e.g., RandomForest) on training data and compute feature importances (MDI / Gini importance).
- Select top-K features (paper found 6 important for CICIoT2023 and 11 for IoTID20).
- *Tip:* `sklearn.ensemble.RandomForestClassifier().feature_importances_`.

Blending ensemble (two-level blending / Voting Estimator meta-model)



🧠 1. They answer **different questions**

▶ LIME answers:

“Which features influenced this prediction the most?”

Example (from IoT traffic):

- High packet rate → prediction toward DDoS
- Large Flow Duration → supports DDoS
- Low IAT variance → reduces confidence

This tells the user **why** the blending model made its decision.

▶ Counterfactual answers:

"What small change would flip the prediction to another class?"

Example:

- If Flow Duration drops by 30%, the result switches from DDoS → Normal
- If SYN flag count reduces, DoS → Benign

This tells the user **what minimal change removes the attack classification**.

🧩 2. They complement each other

Neither method alone gives a full picture.

LIME alone:

- Explains feature importance
- BUT cannot show how to *change* a prediction
- Only describes, not prescribes

Counterfactual alone:

- Shows how to flip the result
- BUT does not show which features were most influential
- No detailed breakdown of local feature contributions

Using both gives:

Understanding + Action

Explanation + Transformation

This is crucial in security.

🔒 3. In IoT IDS, interpretability must be strong

Attack detection is critical, so analysts must know:

✔️ Why an instance is labeled as an attack (LIME)

✔️ How the attack label could be changed or avoided (Counterfactual)

This dual explanation:

- Helps security teams investigate attacks
- Helps network engineers tune thresholds
- Helps researchers understand misclassifications
- Helps justify decisions to non-technical stakeholders

📌 4. Both methods reveal different weaknesses of the model

LIME reveals:

- Features that strongly affect predictions
- Potential biases
- Reasons for misclassifications

Counterfactual reveals:

- Fragile decision boundaries
- How close samples are to other classes
- Cases where the model is overconfident or unstable

Together, they strengthen the paper's claim of **high explainability**.

🎯 5. Your paper's goal is BOTH detection + explanation

The title of your paper is:

> Toward Enhanced Attack Detection and Explanation

Using **two types of explanations** makes the explanation part stronger.

LIME → Local feature-level explanation

Counterfactual → Decision-region level explanation

This aligns perfectly with your research aim.

🏆 Final Answer (short and strong — perfect for a report or slide)

We use **both LIME and Counterfactual explanations** because they complement each other and provide a complete interpretability framework. LIME explains **why** the blending model makes a specific prediction by showing feature contributions, while counterfactual explanations show **how** small changes in input features can flip the prediction to another class. This combination offers deeper insights into the model's decision-making, improves trust, reveals decision boundaries, and supports effective debugging and analysis in IoT intrusion detection systems.

What is a Learning Curve?

A **learning curve** is a graph that shows **how well a machine learning model learns over time** as we give it more training data.

It answers questions like:

- Is the model learning properly?
- Is it overfitting or underfitting?
- Will giving more data improve accuracy?

🌟 Learning Curve has two lines:

1. **Training score** (accuracy/loss on training data)
2. **Validation score** (accuracy/loss on unseen data)

🌈 Simple Example

Imagine training your model on:

- 10% of data → accuracy = 70%
- 50% of data → accuracy = 85%
- 100% of data → accuracy = 94%

Your learning curve shows how accuracy improves as more data is used.

🎯 What Learning Curve Tells You

- If **training accuracy is high** but **validation accuracy is low**

  → the model is **overfitting**
- If **both accuracies are low**

  → the model is **underfitting**
- If **both curves are close and high**

  → the model is **good**

Your paper uses learning curves to show the blending model learns well and generalizes properly.

✅ 2. What is a Confusion Matrix?

A **confusion matrix** is a table that shows the **actual vs predicted** classes of a classification model.

It helps evaluate:

- How many predictions were correct
- Where the model made mistakes
- Which attack types are harder to detect

🌟 Example (Binary Classification)

Classes: Normal, Attack

	Predicted Normal	Predicted Attack
Actual Normal	90 (True Normal)	10 (False Alarm)
Actual Attack	5 (Missed Attack)	95 (Correct)

This tells us:

- 90 normal flows correctly detected
- 10 normal flows wrongly flagged as attacks
- 5 attacks missed
- 95 attacks correctly detected